

Ether, receive, fallback e Checks-Effects-Interactions

La prima volta che il controllo esce dal nostro contratto

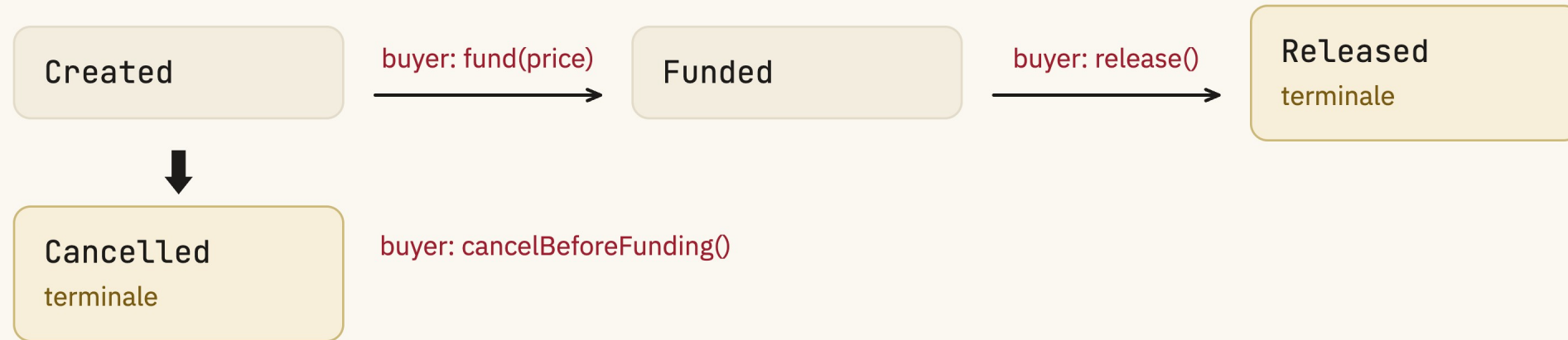
```
seller.call{value: price}("")
```

Contratto didattico, non production-ready: introduciamo una vera interazione esterna, ma la reentrancy completa è la Lezione 5.

Baseline: Solidity 0.8.37
verificata il 21 settembre 2026

DALLA LEZIONE 3

La stessa macchina, ma ora l'Ether esce davvero



LA DIFFERENZA CRUCIALE

La transizione Funded → Released include ora Escrow → CALL con Ether → seller: per la prima volta il controllo può uscire dal nostro contratto.

1 • OBIETTIVI

Cosa saprai fare alla fine della lezione

Come entra Ether

Funzioni payable, receive, fallback e il dispatch; distinguere il balance nativo dal bookkeeping interno del protocollo.

Come esce Ether

call{value: amount}("") al posto di transfer e send, e il controllo obbligatorio del risultato.

Cosa significa cedere il controllo

Una external call è un trasferimento di controllo: high-level contro low-level, e perché success == true non basta.

Checks-Effects-Interactions

Applicarlo, verificarne i limiti, e dimostrare con Foundry che un payout rifiutato reverts senza stato incoerente.

LA DOMANDA DA AUDITOR DI QUESTA LEZIONE

**In quale punto il controllo può lasciare questo contratto,
e quali invarianti devono essere già veri prima che
ciò accada?**

2.1 · ETHER NON È UN ERC-20

Due contabilità diverse, due rischi diversi

ETH: STATO ETHEREUM

address → balance

Ogni account ha un balance mantenuto direttamente dallo stato Ethereum.

```
address(someAccount).balance  
address(this).balance
```

ERC-20: STORAGE DI UN ALTRO CONTRATTO

balances[address] → amount

Il saldo è una voce in un mapping che vive nello storage del contratto token.

La differenza conterà molto quando arriveremo ai token.

Il valore appartiene alla call, non al portafoglio

MSG.VALUE

```
escrow.fund{value: 5 ether}();  
  
// dentro fund():  
msg.sender = buyer  
msg.value  = 5 ether
```

Non è quanto Ether possiede il buyer, né quanto ne possiede il contratto: è **quanto valore nativo è associato al message call corrente**.

PAYABLE

```
function f() external {}           // nonpayable  
function f() external payable {}  // può ricevere
```

payable ≠ deve ricevere Ether
payable = può essere chiamata con Ether

Una funzione payable può essere chiamata anche con msg.value uguale a zero.

2.4 • INVIARE ETHER

Un trasferimento può essere un trasferimento di controllo

IL MODELLO INGENUO

```
Escrow balance -= 5 ETH  
Seller balance += 5 ETH
```

SE IL SELLER È UN CONTRATTO

```
Escrow.release()  
  | CALL(value = 5 ETH)  
  ↓  
Seller.receive()  
  | può eseguire codice  
  | può scrivere storage  
  | può chiamare altri contratti  
  | può tentare una callback verso Escrow  
  ↓  
ritorno a Escrow.release()
```

Vale anche quando il destinatario sembra un normale account utente: dopo EIP-7702 un EOA può avere una delegazione di codice eseguibile. Basare la sicurezza su «questo indirizzo non eseguirà mai codice» è sempre più fragile.

2.5 · I FRAME

Una transazione, molti message call

frame A

`msg.sender = user`

CALL B

frame B

`msg.sender = A`

CALL C

frame C

`msg.sender = B`

Quando C ritorna si torna a B, quando B ritorna si torna ad A. La transazione è una sola, ma al suo interno vivono molti frame di esecuzione: è la base per capire la reentrancy.

3.1 · COME ARRIVA L'ETHER

Tre porte normali, e almeno una strada che le aggira

FUNZIONE PAYABLE NOMINATA

```
fund{value: 5 ether}()
```

CALLDATA VUOTA

```
receive() external payable
```

CALLDATA NON RICONOSCIUTA

```
fallback() external payable
```

MA L'ELENCO NON BASTA PER UN INVARIANT SUL BALANCE ASSOLUTO

Esistono meccanismi EVM che aumentano il balance senza eseguire receive o fallback: il caso didattico è il trasferimento dovuto a SELFDESTRUCT. EIP-6780 ne ha cambiato profondamente la semantica, ma mantiene il trasferimento del balance al beneficiary.

"receive() fa revert" non implica "address(this).balance non potrà mai aumentare"

3.2 · DISPATCH

Quale entry point viene eseguito?



Calldata	value	Entry point tipico
vuota	0	receive(), altrimenti fallback()
vuota	> 0	receive(), altrimenti fallback() se payable
selector valido	0	la funzione corrispondente
selector valido	> 0	la funzione, solo se payable
selector sconosciuto	0	fallback()
selector sconosciuto	> 0	fallback(), solo se payable

1. receive() è selezionata dalla **calldata vuota**, non dalla condizione msg.value > 0.

2. fallback() è anche la rete di sicurezza per i selector non riconosciuti.

Due entry point, due policy esplicite

receive()

```
receive() external payable {  
    revert DirectEtherNotAccepted();  
}
```

Al massimo una, senza nome, senza parametri, senza valori di ritorno, external e payable.

Il funding deve passare da fund(): un receive permissivo sarebbe una seconda porta economica che aggira ruolo, stato e importo.

fallback()

```
fallback() external payable {  
    revert UnknownCall(msg.sig);  
}
```

Esiste anche la forma che riceve la calldata completa e restituisce bytes: non ci serve oggi.

Vogliamo che una call a una funzione inesistente sia rifiutata rumorosamente: il fallback non è una API generica.

Sono scelte di **policy della API**, non invarianti assoluti sul raw balance.

3.5 · CONTABILITÀ

Raw balance e protocol accounting non sono la stessa cosa

LA TENTAZIONE

```
uint256 public accountedEther;  
  
receive() external payable {  
    accountedEther += msg.value;  
}  
  
accountedEther = address(this).balance
```

Non è una proprietà generale dell'EVM: se l'Ether arriva senza eseguire receive, il balance cresce e il contatore no.

RAW BALANCE

Quanti wei si trovano fisicamente all'indirizzo.

PROTOCOL ACCOUNTING

Quanti wei il protocollo considera appartenenti a quali obbligazioni o crediti.

Tornerà in vault, lending e token accounting.

Il risultato va controllato, sempre

```
(bool success, bytes memory returnData) =  
    recipient.call{value: amount}("");  
  
// pattern minimo  
(bool success,) = recipient.call{value: amount}("");  
if (!success) revert EtherTransferFailed();
```

PAYLOAD VUOTO

Stiamo chiedendo una plain call con valore: se il recipient è un contratto con receive, normalmente entriamo lì.

CALL INOLTRA CONTROLLO

Prima della riga il controllo è nostro. Durante la riga passa al recipient, che può essere un EOA, un contratto con receive o fallback, o un account con comportamento delegato.

3.7 · PERCHÉ NON USIAMO TRANSFER()

Lo stipend di gas non è un confine di sicurezza

IL VECCHIO RIFLESSO

```
recipient.transfer(amount);  
bool ok = recipient.send(amount);
```

Legate a uno stipend di 2300 gas, con l'idea implicita: «se inoltre poco gas, il receiver non potrà diventare pericoloso».

È un confine fragile: i costi in gas degli opcode possono cambiare. La documentazione Solidity marca send e transfer come **deprecati e pianificati per la rimozione**.

QUELLO CHE USEREMO

```
call{value: amount}("")
```

La conseguenza, accettata consapevolmente: il recipient può ricevere gas a sufficienza ed eseguire codice. La sicurezza deve venire dalla **progettazione del nostro stato**, non dall'aspettativa che il receiver sia troppo povero di gas per comportarsi in modo complesso.

Chi si prende la responsabilità del significato

HIGH-LEVEL EXTERNAL CALL

```
IERC20(token).transfer(to, amount);
```

Il compilatore conosce la firma ABI: type checking, encoding, decode del ritorno, propagazione dei revert.

LOW-LEVEL CALL

```
(bool success, bytes memory data) =  
    target.call(payload);
```

Più vicina alla primitive EVM: calldata, returndata, controllo di success e validazione del target sono responsabilità tua.

SUCCESS == TRUE NON È SUCCESSO DI BUSINESS

Significa che il frame di chiamata non è terminato con una failure EVM. Se il target non ha la funzione ma possiede un fallback permissivo che non reverte, la call può tornare true senza che la funzione voluta sia stata eseguita.

3.11 · ATOMICITÀ

Se il payout fallisce, l'Effect viene annullato con tutto il resto

```
state = State.Released;

(bool success,) = seller.call{value: price}("");

if (!success) {
    revert EtherTransferFailed(seller, price);
}
```

L'OBIEZIONE NATURALE

«Se impostiamo Released prima e poi il payout fallisce, non resta uno stato falso?»

```
state = Released
    ↓
external call fallisce
    ↓
revert
    ↓
state torna Funded
balance torna quello precedente
```

Un revert annulla gli effetti persistenti di quel percorso di esecuzione: è ciò che permette di applicare CEI senza accettare uno stato finale incoerente.

3.12 · CHECKS-EFFECTS-INTERACTIONS

Prepara lo stato, poi cedi il controllo

1 · CHECKS

chi sta chiamando?
lo stato permette l'azione?
gli input sono validi?
l'importo è valido?

2 · EFFECTS

state variables
balance interni
crediti
nonce, flag

3 · INTERACTIONS

CALL
funzione di un altro contratto
trasferimento ETH
callback, hook

L'IDEA DIFENSIVA

Se il codice esterno tenta di rientrare, deve osservare lo stato già aggiornato, non quello vecchio che permetteva l'azione originale.

3.13 · CEI NON È UNA FORMULA MAGICA

Un pattern importante, non una prova di correttezza

- Sono stati aggiornati tutti gli effetti critici prima dell'interazione?
- Esistono altre funzioni che il callback può chiamare?
- Lo stato di altri contratti rimane coerente?
- Ci sono callback indirette o hook token?
- Esistono invariant che coinvolgono più contratti?
- L'external call può fallire e bloccare il flusso?

`external call = trust boundary`

Lo stato interno deve essere preparato **prima** di attraversarla.
Nella Lezione 5 vedremo esattamente perché non basta.

3.14 · PUSH E PULL

Due famiglie di design per il pagamento

PUSH · QUELLO DI OGGI

```
buyer → release()  
      ↓  
Escrow paga il seller subito
```

Rende evidente la external call. Ma se il recipient rifiuta Ether l'azione può fallire, e il recipient entra nel control flow della funzione.

PULL · IL CREDITO

```
release()  
      ↓  
claimable[seller] += amount  
  
seller → withdraw()
```

Riduce il coupling tra la transizione di stato e il payout. OWASP e la documentazione Solidity raccomandano spesso design withdrawal/pull.

Oggi usiamo **intenzionalmente** un push payout, perché vogliamo osservare la trust boundary. Non è una dichiarazione che il push sia la scelta migliore per un protocollo reale.

Gli stati, gli errori nuovi, la configurazione

```
enum State {
    Created, Funded, Released, Cancelled
}

error Unauthorized(address caller);
error WrongState(State expected, State actual);
error WrongValue(uint256 expected, uint256 actual);
error EtherTransferFailed(
    address recipient, uint256 amount);
error DirectEtherNotAccepted();
error UnknownCall(bytes4 selector);
```

```
event Funded(address indexed buyer,
    uint256 amount);
event Released(
    address indexed buyer,
    address indexed seller,
    uint256 amount
);
event Cancelled(address indexed buyer);

address public immutable buyer;
address payable public immutable seller;
uint256 public immutable price;

State public state;
```

Tre errori nuovi rispetto alla Lezione 3, e un tipo nuovo: il seller è **address payable**, perché ora riceve davvero.

4 • LA FUNZIONE DELLA LEZIONE

release: Checks, Effects, Interactions

```
function release() external onlyBuyer onlyState(State.Funded) {  
    // EFFECT: consumiamo la transizione  
    state = State.Released;  
  
    // INTERACTION: il controllo può passare al seller  
    (bool success,) = seller.call{value: price}("");  
  
    // una low-level call non propaga la failure  
    if (!success) {  
        revert EtherTransferFailed(seller, price);  
    }  
  
    emit Released(buyer, seller, price);  
}
```

1 • CHECKS

nei modifier

2 • EFFECTS

prima della boundary

3 • INTERACTION

qui il controllo esce

4 • ENFORCEMENT

false → revert

4 · LE DUE PORTE

Ether diretto e selector sconosciuti: rifiutati per policy

```
/// Il funding diretto non è parte dell'API.  
receive() external payable {  
    revert DirectEtherNotAccepted();  
}  
  
/// Selector sconosciuti non fanno parte dell'API.  
fallback() external payable {  
    revert UnknownCall(msg.sig);  
}
```

FUND() E CANCELBEFOREFUNDING()

Restano quelle della Lezione 3: prezzo esatto da Created, cancellazione solo prima del funding.

Entrambe le porte sono **payable**: una call con selector sconosciuto può portare Ether, ma il body reverte comunque.

5.2 • ANALISI DI RELEASE()

Le domande da auditor, applicate alla funzione chiave

Domanda	Risposta
Chi può chiamarla?	Solo il buyer, grazie a onlyBuyer
Input del chiamante	Nessun parametro: ma il caller sceglie quando chiamare, e la sequenza è essa stessa un input
Valore in ingresso	Nessuno: la funzione è nonpayable
Stato letto	buyer, state, seller, price
Stato modificato	state = Released, prima dell'interazione
Chiamate esterne	seller.call{value: price}("") — l'unica riga in cui il controllo esce
Assunzioni	Nessuna sul seller: contratto, receive, storage, altre call, revert — tutto è possibile

Il comportamento ricorsivo del seller — può richiamare l'Escrow durante la call? — è il laboratorio della Lezione 5.

5.4 · SE IL SELLER RIFIUTA ETHER

Il fallimento diventa un revert, e lo stato torna indietro

```
success = false
  ↓
revert EtherTransferFailed(seller, price)
  ↓
rollback dell'intera operazione
```

STATO FINALE

```
state          = Funded
Escrow balance = price
seller balance  invariato
```

È esattamente il comportamento che vogliamo per questa specifica: nessun payout, nessuna transizione.

Il recipient però può **bloccare** la transizione: è uno dei motivi per cui i design reali spesso preferiscono un pull payment.

Autorizzazione, macchina a stati, economia

AUTORIZZAZIONE

- solo il buyer può fund()
- solo il buyer può release()
- solo il buyer può cancelBeforeFunding()

STATE MACHINE

- fund() solo da Created
- release() solo da Funded
- cancelBeforeFunding() solo da Created
- una release riuscita porta a Released
- Released e Cancelled sono terminali

ECONOMICHE

- un funding riuscito richiede esattamente price
- una release riuscita trasferisce esattamente price al seller
- il seller non deve ricevere due payout

L'ultima — nessun doppio payout — diventerà particolarmente importante nella Lezione 5.

Atomicità applicativa, e ciò che non dobbiamo promettere

SE IL PAYOUT FALLISCE

- `release()` deve revertire
- lo stato deve rimanere Funded
- i price wei devono rimanere nell'Escrow

Sono proprietà di **atomicità applicativa**.

CALL DIRETTE

- una plain call con calldata vuota deve fallire
- una call con selector sconosciuto deve fallire

MA NON SCRIVIAMO

«Il balance dell'Escrow può aumentare soltanto tramite `fund()`»: l'EVM permette trasferimenti che non passano dal dispatch del recipient.

6.6 · L'INVARIANT CONTABILE

Formularlo in modo che l'Ether inatteso non lo rompa

INGENUO

```
state = Funded  
⇒ address(escrow).balance = price
```

Può essere rotto da Ether forzato senza che la logica del protocollo sia stata violata.

PIÙ ROBUSTO

```
state = Funded  
⇒ address(escrow).balance ≥ price
```

In parole: se lo stato è Funded, il protocollo deve essere in grado di pagare almeno price.

LA SEPARAZIONE CHE CONTA

```
protocol liability = price                      contro                      raw balance = address(escrow).balance
```

Un surplus inatteso non deve creare un nuovo credito per il seller.

Sette domande sul destinatario



Domanda	Risposta
Il seller può essere un contract?	Sì
Può rifiutare Ether?	Sì
Può eseguire codice durante il payout?	Sì
Può effettuare altre call?	Sì
Deve essere trusted perché il protocollo sia corretto?	Idealmente no, per gli invariant fondamentali
Se revert, cosa succede?	La release revert e lo stato resta Funded
Cosa vede se fa una callback durante release?	State.Released, perché gli Effects vengono prima

L'ultima riga è la porta d'ingresso alla Lezione 5.

Tre destinatari giocattolo, tre comportamenti

AcceptingSeller

```
receive() external payable {  
    totalReceived += msg.value;  
}
```

Dimostra che il payout non è solo un aggiornamento di balance: il recipient può eseguire uno SSTORE.

RejectingSeller

```
receive() external payable {  
    revert("REJECT_ETH");  
}
```

Il destinatario che rifiuta: farà fallire la call e quindi revertire la release.

ForceEther

```
function force(  
    address payable t)  
    external {  
        selfdestruct(t);  
    }
```

Trasferisce il balance senza passare da receive: serve solo a dimostrare una proprietà dell'EVM.

SELFDESTRUCT è deprecato e non va preso come pattern applicativo. Qui è solo uno strumento didattico locale: EIP-6780 ne ha cambiato la semantica di distruzione, ma mantiene il trasferimento del balance al target.

8.3 · IL PAYOUT CHE RIESCE

Prima un EOA, poi un contratto che esegue codice

```
function testReleaseToContractExecutesReceiverCode() public {
    AcceptingSeller receiver = new AcceptingSeller();

    // un Escrow il cui seller è il contratto
    _fund(contractSellerEscrow);
    assertEq(receiver.totalReceived(), 0);

    vm.prank(buyer);
    contractSellerEscrow.release();

    assertEq(receiver.totalReceived(), PRICE);
    assertEq(address(contractSellerEscrow).balance, 0);
}
```

TESTRELEASEPAYSEOASELLER

Stato Released, balance dell'Escrow a zero, saldo del seller cresciuto di PRICE.

Il secondo test è la prova del modello mentale: **totalReceived passa da 0 a PRICE**, quindi durante il payout è stato eseguito codice del destinatario.

8.3 · IL PAYOUT CHE FALLISCE

Il test di regressione più importante della lezione

```
function testRejectingSellerMakesReleaseRevertAtomically() public {
    RejectingSeller receiver = new RejectingSeller();
    // ... escrow con quel seller, poi funding

    vm.expectRevert(abi.encodeWithSelector(
        EscrowWithPayout.EtherTransferFailed.selector,
        address(receiver), PRICE));
    vm.prank(buyer);
    rejectingEscrow.release();

    // l'effetto State.Released è stato rollbackato
    assertEq(uint256(rejectingEscrow.state()), uint256(...Funded));
    assertEq(address(rejectingEscrow).balance, PRICE);
    assertEq(address(receiver).balance, 0);
}
```

COSA DIMOSTRA

Il revert attraversa la trust boundary e annulla anche l'Effect scritto prima della call.

L'errore è verificato con **tutto il revert data**: selector, recipient e importo.

8.3 · LE PORTE CHIUSE

Con una low-level call si controlla il boolean, non expectRevert

```
function testPlainEtherTransferIsRejected() public {
    vm.prank(outsider);
    (bool success,) = address(escrow).call{value: 1 ether}("");

    assertFalse(success);
    assertEquals(address(escrow).balance, 0);
}

function testUnknownSelectorIsRejected() public {
    bytes memory unknownCall = hex"deadbeef";
    vm.prank(outsider);
    (bool success,) = address(escrow).call(unknownCall);

    assertFalse(success);
}
```

PERCHÉ COSÌ

Il revert del receiver torna al chiamante come success = false: la call di test non propaga il revert, quindi lo verifichiamo sul boolean.

COSA RESTA INVARIATO

Balance a zero e stato ancora Created: le porte chiuse non muovono nulla.

8.3 · ETHER FORZATO

Il raw balance cresce, la macchina a stati no

```
function testForcedEtherBypassesReceive() public {
    vm.deal(address(this), 1 ether);
    ForceEther force = new ForceEther{value: 1 ether}();

    // non chiama fund(), non esegue receive()
    force.force(payable(address(escrow)));

    assertEq(address(escrow).balance, 1 ether);
    assertEq(uint256(escrow.state()), uint256(...Created));
}
```

TESTFORCEDETHERDOESNOTINCREASES ELLERENTITLEMENT

Con il surplus già dentro, il funding aggiunge PRICE. Dopo la release il seller riceve **solo PRICE** e il surplus resta nel contratto.

Non è un exploit contro il nostro Escrow: è una proprietà della piattaforma. Per un auditor cambia **come si scrivono gli invariant**.

Le tre tracce da guardare in aula

PAYOUT VERSO UN CONTRATTO

```
forge test \
  --match-test testReleaseToContractExecutesReceiverCode \
  -vvvv
```

```
EscrowWithPayout::release()
→ AcceptingSeller::receive()
```

PAYOUT RIFIUTATO

```
buyer
↓
Escrow.release()
| state = Released
↓
RejectingSeller.receive() → revert
↓
success = false
↓
Escrow reverts EtherTransferFailed
↓
state = Funded · balance = PRICE
```

La terza traccia da guardare è quella di **testForcedEtherBypassesReceive**: il balance cambia senza che compaia una chiamata al receive dell'Escrow.

9 • TEST NEGATIVI

Undici cose da dimostrare, non una percentuale

Scenario	Atteso
outsider release	revert
release prima del funding	revert
release due volte	revert
cancel dopo il funding	revert
plain ETH transfer	failure
selector sconosciuto	failure
seller che rifiuta ETH	release revert
payout fallito	lo stato resta Funded
payout fallito	l'ETH resta nell'Escrow
Ether forzato	non cambia la state machine
Ether forzato	non aumenta il payout dovuto

Questa lista è già più utile di una metrica astratta come «abbiamo il 90% di code coverage».

La coverage dice quali righe sono state eseguite. Non dimostra che hai formulato le proprietà giuste.

L'ordine CEI è rispettato, e il contratto è comunque rotto

```
function release() external {
    require(msg.sender == buyer, "NOT_BUYER");
    require(state == State.Funded, "WRONG_STATE");

    state = State.Released;

    (bool success,) = seller.call{value: price}("");

    // BUG: registriamo il risultato, ma non revertiamo
    emit PayoutAttempted(success);
}
```

EFFECTS PRIMA DI INTERACTION

L'ordine è quello giusto: CEI è rispettato alla lettera.

MA

CEI protegge una classe di problemi di sequencing. Non trasforma una call fallita in una call riuscita.

La transaction riesce, il seller non è stato pagato

```
function testUncheckedCallCreatesFalseReleasedState() public {
    RejectingSeller receiver = new RejectingSeller();
    // broken = UncheckedCallEscrow con quel seller

    vm.prank(buyer); broken.fund{value: PRICE}();
    vm.prank(buyer); broken.release();

    // la transaction non è revertita...
    assertEq(uint256(broken.state()), uint256(...Released));

    // ...ma il seller non è stato pagato
    assertEq(address(receiver).balance, 0);
    assertEq(address(broken).balance, PRICE);
}
```

PROPRIETÀ VIOLATA

«Una release riuscita deve trasferire esattamente price al seller.»

Lo stato racconta una **storia falsa**:
Released, seller payout zero, price
ancora bloccato nel contratto.

Il risultato della call va fatto rispettare

LA CAUSA NON È «CALL È CATTIVA»

```
external call può fallire
+
risultato non fatto rispettare
+
stato applicativo la considera conclusa
```

IN FORMA DI AUDIT FINDING

The low-level ETH transfer result is not enforced. A reverting recipient leaves the escrow in Released state while the seller receives no funds and the ETH remains locked.

LA CORREZIONE

```
state = State.Released;

(bool success,) = seller.call{value: price}("");

if (!success) {
    revert EtherTransferFailed(seller, price);
}
```

Il test di regressione è
testRejectingSellerMakesReleaseRevertAtomically:
se qualcuno rimuove il check su success, deve diventare rosso.

Lo stesso payout fallito, due esiti diversi

PRIMA · RISULTATO IGNORATO

state = Released
seller paid = no
escrow balance = price
transaction = success

Lo stato applicativo è incoerente.

DOPO · PAYOUT FALLITO

state = Funded
seller paid = no
escrow balance = price
transaction = revert

Nessuna transizione, nessun fondo perso.

DOPO · PAYOUT RIUSCITO

state = Released
seller paid = yes, price
escrow balance = surplus
transaction = success

Non assumiamo che il balance iniziale fosse esattamente price.

Marca subito ogni operazione che può cedere il controllo

EVIDENZIALE TUTTE

target.call(...)

target.delegatecall(...)

target.staticcall(...)

IContract(target).someFunction(...)

IERC20(token).transfer(...)

IERC721(...).safeTransferFrom(...)

Non hanno la stessa semantica, ma meritano tutte la stessa domanda: **può eseguire codice che non controllo?**

POI GUARDA COSA VIENE SCRITTO DOPO

```
externalInteraction();
```

```
balances[user] = 0;  
state = Completed;  
used[id] = true;
```

«Perché queste Effects avvengono dopo aver ceduto il controllo?» Non è automaticamente una vulnerabilità: è un punto da investigare.

Come entra l'Ether, come esce

Ingresso di Ether

- Quali funzioni sono payable?
- Il contratto definisce receive()? E fallback()?
- Il fallback è payable?
- L'Ether inviato direttamente ha una semantica definita o viene rifiutato?
- La logica assume che tutto l'Ether passi da una funzione specifica?
- Esistono invariant fragili del tipo `balance == accounting`?

Uscita di Ether

- Dove viene effettuato il value transfer?
- Viene usata una low-level call?
- Il return value viene controllato?
- Se il recipient fa revert, quale stato resta?
- Il recipient può bloccare una transizione importante?
- È più appropriato un pull payment?

Il controllo esterno, l'API, i test

Controllo esterno

- Qual è l'ultima modifica di storage prima della external call?
- Tutti gli effetti critici sono già applicati?
- Il callback può osservare stato intermedio?
- Può chiamare un'altra funzione del contratto?
- Può attraversare altri contratti e tornare indietro?

API surface

- I selector sconosciuti vengono accettati o rifiutati?
- `receive()` e `fallback()` fanno più lavoro del necessario?
- Esiste una seconda strada accidentale per depositare fondi?

Testing

- Hai testato un recipient EOA?
- Un recipient contract?
- Un recipient che fa revert?
- Hai verificato il rollback dello stato?
- Il raw balance dopo un fallimento?
- L'Ether inatteso, quando gli invariant dipendono dal balance?

Sette frasi da non ripetere

1 · «receive() viene chiamata solo se msg.value > 0»

No: la discriminante è la calldata vuota.

2 · «Se receive() reverte, il contratto non può ricevere Ether»

Troppo forte: il raw balance può crescere in altri modi.

3 · «Trasferire Ether è solo spostare numeri»

No: una CALL verso un contract può eseguire codice.

4 · «Uso transfer(), quindi la reentrancy è risolta»

Difesa obsoleta e fragile, ora deprecata.

5 · «Ho usato CEI, quindi è tutto risolto»

UncheckedCallEscrow rispetta CEI ed è rotto.

6 · «Se .call() torna true, la funzione è stata eseguita»

True riguarda il frame EVM, non la tua intenzione.

7 · «Un EOA non esegue codice»

Con EIP-7702 un EOA può eseguire codice delegato.

La regola difensiva

Se cedi controllo a un address, progetta come se quel percorso potesse eseguire codice non fidato.

Prevedi, poi verifica in locale

1 • Dispatch matrix

Con `foo()` payable, `receive` e `fallback`: prevedi l'entry point per sei combinazioni di calldata e valore, poi verifica con eventi.

2 • Nonpayable

Rendi `foo()` nonpayable: un test con 0 wei passa, uno con 1 wei reverte. Dove avviene il rifiuto rispetto al body?

3 • Payout rifiutato

Un receiver che reverte con custom error: `release` reverte, stato `Funded`, balance `PRICE`, receiver a zero.

4 • Ether inatteso

`forcedAmount` 13 ether con `PRICE` 5: perché sarebbe pericoloso pagare `address(this).balance` invece dell'obbligazione?

5 • Trova il bug

`release()` che manda l'intero balance e scrive lo stato dopo: quattro osservazioni su ordering, amount, trust boundary, test mancante.

6 • Pull design

Ridisegna in pseudocodice: `claimable[seller]` e `withdraw()`. Nuove proprietà, nuove variabili, dove sta la trust boundary.

Nove idee da portare via

1 · msg.value appartiene alla call corrente

Non è il balance dell'utente né quello del contratto.

2 · payable abilita l'ingresso di Ether

Su quella call: non significa che debba riceverne.

3 · receive e fallback sono superficie di attacco

Sono entry point, vanno progettati intenzionalmente.

4 · Raw balance ≠ protocol accounting

Non assumere che ogni wei sia passato dal tuo deposito.

5 · Una external call è un trasferimento di controllo

call = trust boundary.

6 · Le low-level call vanno controllate

if (!success) revert ...

7 · CEI: Checks → Effects → Interactions

Lo stato locale coerente prima di cedere controllo.

8 · Il revert dopo una interaction fallita rollbacka gli Effects

È ciò che riporta release a Funded.

9 · CEI non basta per ogni forma di reentrancy

La Lezione 5 parte da questa limitazione.

Dovresti saper spiegare tutto questo senza appunti

✓ differenza tra `msg.value` e `address(this).balance`

✓ cosa significa payable

✓ quando scatta `receive()`

✓ quando scatta `fallback()`

✓ perché `receive()` non è una garanzia sul raw balance

✓ cosa restituisce una low-level call

✓ perché `success` va controllato

✓ perché una call con Ether può eseguire codice

✓ cosa significa trust boundary

✓ ordine Checks → Effects → Interactions

✓ perché il rollback salva lo stato se il payout fallisce

✓ perché CEI non equivale a «sicuro per definizione»

✓ differenza tra push e pull payment

✓ perché `address(this).balance` non definisce il payout dovuto

Se uno di questi punti non è chiaro, conviene consolidarlo prima di affrontare la reentrancy.

Da dove viene ogni affermazione

Fonte	Per che cosa
Solidity 0.8.37 Release Announcement	baseline del compilatore
Solidity · Receive Ether Function · Fallback Function	sintassi, dispatch, requisiti external e payable
Solidity · Members of address types	balance, call, send, transfer e la loro deprecazione
Solidity · Security Considerations	pericoli delle external call, CEI, withdrawal pattern
Foundry · Reference · cheatcodes · expectRevert	forge test, vm.prank, expectRevert con low-level call
EIP-6780	semantica moderna di SELFDESTRUCT e trasferimento del balance
EIP-7702	un EOA può eseguire codice delegato
OWASP · SC08 Reentrancy · SCWE-102	unchecked external call, CEI, pull withdrawals
OpenZeppelin Contracts 5.x · Utilities	gestione dei value transfer, riferimento secondario